Analysis of Algorithm
Assignment no 1

Name : Arslan Tahir
SAP : 47120
Section: SE 4-1 ; 05/10/26
Github: https://github.com/Sapid47120/Fall26AOA2026

Q1 Algorithm Design and Asymptotic Analysis.

1 Algorithmic Strategy.

We use linear traversal Strategy. We Scan the array from the first element to the last element while maintaining two variables: min and max. Initially both are set to the first element of the array. For each remaining element. we compare it with min and max. If the element is smaller than min. We update min if its larger than max we update max After scanning the complete array min contains the smallest element and max contains the largest element.

2) Pseudocode

Scanned with CamScanner

Arslan Ahmed
47120

Algorithm FindMinMax(A, n)

Input: An array of A of n integers
Output: The smallest and largest elements.

```
1.  min ← A[1]
2.  min ← A[1]

3   FOR i ← TO n DO
4   IF A[i] < min Then
5         min ← A[i]
6       End if

7   IF A[i] > max Then
8       max ← A[i]
a     END IF
10  End For
11   return min, max.
```

3: Major primitive operations.

1: Assignment initializing & updating min and max
2: Comparison comparing A[i] with min and max
3: Array Access accessing each element A[i]
4: loop iteration repeating the operations of each element.

Scanned with CamScanner

4 Best case and worst case Running Time

- BestCase: The Algorithm still scans all n elements because it must examine every element to guvantee finding both the smallest and largest values Therefore, the best-case running time is Θn.

- Worst Case: The Algorithm also scans all all n elements and performs the required comparison for each element therefore the worst running time is Θn.

Conclusion is:

- Best case : Θn
- Worst case: Θn

5 Asymptotic Analysis.

- Time Complexity On:

- Big-O : $O(n)$
- Big-Ω : $\Omega(n)$
- Big-Θ $\Theta(n)$

Reason: The Agorithm examines every element of an Array, so its running time grows linearly with n

Scanned with CamScanner

6 Space Complexity: The Algorithm uses only two additional variables min and max and the loop variable i it does not require any additional array or data structure. Therefore the auxiliary space complexity is O(1).

Q2: Design & compare Algorithm

7) Two different Algorithm Strategies

Algorithm 1: linear Search

The array is searched sequentially from the first element to the last element. Each element is compared with x. If an element equal x, the algorithm return x exist If the entire array is searched without finding x. it return that x does not exist.

Algorithm 2: Binary Search.

It is used when the array is Sorted ~~Stored~~ The middle element is compared with x if it is equal to x the search is

Scanned with CamScanner

Successfull. If x is smaller the left half is searched; Otherwise the right half is searched. This process continues until x is found or the the search range becomes empty.

8 Pseudocode.

Algorithm 1 : linear Search

Algorithm linearSearch (A, n, x)
Input: An array A of n integers and value x.

Output: whether x exists in A.

```
1  For i ← 1 To n DO
2      IF A[i] = x Then
           Return TRUE
4      End IF
5  End FOR
6  Return FAISE.
```

Algorithm 2: Binary Search

Algorithm binary Search (A, n, x)

Scanned with CamScanner

Assignment 4,7,10

Input : A Sorted Array A of n integers and value x

Output : Whether x exist in A

```
1   low ← 1
2   high ← n
3   While low ≤ high DO
4   mid ← floor((low + high) / 2
5   IF A[mid] = x Then
6   Return TRUE
7   ELSE IF x < A[mid] Then
8   high ← mid - 1
9   ELSE
10      low ← mid + 1
11  End IF
12  End while
13  Return False.
```

9) Best CASE and Worst Case Scenario Time complexity.

7) Linear Search

- Best CASE : O(1) - x is found at the first position
- Worst CASE : O(n) - x is at the last position or does not exist

Scanned with CamScanner

9. Binary Search:

- Best CASE: O(1) - x is found at the middle position in the first comparison
- Worst CASE: O(log n) - the search range is repeatedly divided in half until the element is found or the range becomes empty.

10. Asymptotic Analysis

(1) linear Search:

- Big-O (upper bound): O(n)
- Big-Ω (lower bound): Ω(1)
- Big-Θ (Tight Bound): Θ(n)

(2) Binary Search:

- Big-O: (upper-bound): O(log n)
- Big-Ω: (lower-bound): Ω(1)
- Big-Θ: (Tight bound): O(log n)

11. Comparison: linear Search is suitable when array is unsorted or when the input size is small. It does not require any preprocessing. Binary Search is more

Scanned with CamScanner

efficent for large arrays because it reduces the search space by half at each step. but the array must be sorted, . therefore. For a large sorted array Binary Search is generally more efficient than linear Search.

12 : Input Size and Arrangement.

For a small array linear Search may be sufficent. because its implementation is simple and does not require the array to be sorted. For a large unsorted array. linear Search may require checking many elements. If the array is sorted, Binary Search is preferable because it reduces the Search space by half at every step. Therefore the Size and arrangement of the input affect which Algorithm is more suitable.

END.

Scanned with CamScanner